Nodes and Constraints

```
// FESTIVAL
CREATE CONSTRAINT `festival_id_Festival_uniq` IF NOT EXISTS
FOR (n: `Festival`)
REQUIRE (n.`festival_id`) IS UNIQUE;

UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`festival_id` IN $idsToSkip AND NOT
toInteger(trim(nodeRecord.`festival_id`)) IS NULL
MERGE (n: `Festival` { `festival_id`:
toInteger(trim(nodeRecord.`festival_id`)) })
SET n.`name` = nodeRecord.`name`
SET n.`location` = nodeRecord.`location`
SET n.`startDate` = datetime(nodeRecord.`startDate`)
SET n.`endDate` = datetime(nodeRecord.`endDate`);

// STAGE
CREATE CONSTRAINT `stage_id_Stage_uniq` IF NOT EXISTS
FOR (n: `Stage`)
REQUIRE (n.`stage_id`) IS UNIQUE;

UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`stage_id` IN $idsToSkip AND NOT
toInteger(trim(nodeRecord.`stage_id`)) IS NULL
MERGE (n: `Stage` { `stage_id`:
toInteger(trim(nodeRecord.`stage_id`)) })
SET n.`name` = nodeRecord.`name`
SET n.`maxCapacity` = toInteger(trim(nodeRecord.`maxCapacity`))
SET n.`locationDescription` = nodeRecord.`locationDescription`;

// PERFORMANCE
CREATE CONSTRAINT `performance_id_Performance_uniq` IF NOT EXISTS
FOR (n: `Performance`)
REQUIRE (n.`performance_id`) IS UNIQUE;

UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`performance_id` IN $idsToSkip AND NOT
toInteger(trim(nodeRecord.`performance_id`)) IS NULL
MERGE (n: `Performance` { `performance_id`:
toInteger(trim(nodeRecord.`performance_id`)) })
SET n.`title` = nodeRecord.`title`
SET n.`genre` = nodeRecord.`genre`
```

```
SET n.`startTime` = datetime(nodeRecord.`startTime`)
SET n.`endTime` = datetime(nodeRecord.`endTime`);

// SOLOMUSICIAN

// Each SoloMusician is an Artist (Total & Exclusive ISA
Hierarchy)
// The uniqueness of artist_id is enforced globally on :Artist
// Constraint on :SoloMusician is optional and redundant, so
omitted here

UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`artist_id` IN $idsToSkip AND NOT
toInteger(trim(nodeRecord.`artist_id`)) IS NULL
MERGE (n: SoloMusician: Artist{ `artist_id`:
toInteger(trim(nodeRecord.`artist_id`)) })
SET n.`country` = nodeRecord.`country`
SET n.`genre` = nodeRecord.`genre`
SET n.`stageName` = nodeRecord.`stageName`
SET n.`fullName` = nodeRecord.`fullName`
SET n.`nationality` = nodeRecord.`nationality`
SET n.`instrument` = nodeRecord.`instrument`;

// BAND

// Each Band is an Artist (Total & Exclusive ISA Hierarchy)
// The uniqueness of artist_id is enforced globally on :Artist
// Constraint on :Band is optional and redundant, so omitted here

UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`artist_id` IN $idsToSkip AND NOT
toInteger(trim(nodeRecord.`artist_id`)) IS NULL
MERGE (n: Band: Artist{ `artist_id`:
toInteger(trim(nodeRecord.`artist_id`)) })
SET n.`country` = nodeRecord.`country`
SET n.`genre` = nodeRecord.`genre`
SET n.`stageName` = nodeRecord.`stageName`
SET n.`formationYear` =
toInteger(trim(nodeRecord.`formationYear`))
SET n.`bandName` = nodeRecord.`bandName`;

// ARTIST

// Superclass label and global uniqueness constraint
CREATE CONSTRAINT `artist_id_Artist_uniq` IF NOT EXISTS
FOR (n:Artist)
```

```
REQUIRE (n.artist_id) IS UNIQUE;

// Ensure Total & Exclusive (diagnostic checks)
// Should return no results if the model is consistent

// 1. Total check: all Artists must be either Band or SoloMusician
MATCH (a:Artist)
WHERE NOT (a:SoloMusician) AND NOT (a:Band)
RETURN a LIMIT 5;

// 2. Exclusive check: no node should be both Band and
SoloMusician
MATCH (a:Artist:SoloMusician:Band)
RETURN a LIMIT 5;


// STAND
CREATE CONSTRAINT `stand_id_Stand_uniq` IF NOT EXISTS
FOR (n: `Stand`)
REQUIRE (n.`stand_id`) IS UNIQUE;

UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`stand_id` IN $idsToSkip AND NOT
toInteger(trim(nodeRecord.`stand_id`)) IS NULL
MERGE (n: `Stand` { `stand_id`:
toInteger(trim(nodeRecord.`stand_id`)) })
SET n.`standName` = nodeRecord.`standName`
SET n.`sponsor` = nodeRecord.`sponsor`;


// PRODUCT
CREATE CONSTRAINT `barcode_Product_uniq` IF NOT EXISTS
FOR (n: `Product`)
REQUIRE (n.`barcode`) IS UNIQUE;

UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`barcode` IN $idsToSkip AND NOT
toInteger(trim(nodeRecord.`barcode`)) IS NULL
MERGE (n: `Product` { `barcode`:
toInteger(trim(nodeRecord.`barcode`)) })
SET n.`name` = nodeRecord.`name`
SET n.`category` = nodeRecord.`category`
SET n.`price` = toFloat(trim(nodeRecord.`price`));

// TICKET
CREATE CONSTRAINT `ticket_id_Ticket_uniq` IF NOT EXISTS
```

```
FOR (n: `Ticket`)
REQUIRE (n.`ticket_id`) IS UNIQUE;

UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`ticket_id` IN $idsToSkip AND NOT
toInteger(trim(nodeRecord.`ticket_id`)) IS NULL
MERGE (n: `Ticket` { `ticket_id`:
toInteger(trim(nodeRecord.`ticket_id`)) })
SET n.`type` = nodeRecord.`type`
SET n.`price` = toFloat(trim(nodeRecord.`price`))
SET n.`startValidity` = datetime(nodeRecord.`startValidity`)
SET n.`endValidity` = datetime(nodeRecord.`endValidity`);

// CUSTOMER
CREATE CONSTRAINT `customer_id_Customer_uniq` IF NOT EXISTS
FOR (n: `Customer`)
REQUIRE (n.`customer_id`) IS UNIQUE;

UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`customer_id` IN $idsToSkip AND NOT
toInteger(trim(nodeRecord.`customer_id`)) IS NULL
MERGE (n: `Customer` { `customer_id`:
toInteger(trim(nodeRecord.`customer_id`)) })
SET n.`fullName` = nodeRecord.`fullName`
SET n.`email` = nodeRecord.`email`
SET n.`phone` = nodeRecord.`phone`;

// ARTIST
CREATE CONSTRAINT `artist_id_Artist_uniq` IF NOT EXISTS
FOR (n: `Artist`)
REQUIRE (n.`artist_id`) IS UNIQUE;

// CREATING LABEL ARTIST

// Add :Artist to subtypes to represent the Total and Exclusive
ISA hierarchy
MATCH (b:Band)
SET b:Artist;

MATCH (s:SoloMusician)
SET s:Artist;

// Ensure Total & Exclusive (diagnostic checks)
// Should return no results if the model is consistent

// 1. Total check: all Artists must be either Band or SoloMusician
```

```
MATCH (a:Artist)
WHERE NOT (a:SoloMusician) AND NOT (a:Band)
RETURN a LIMIT 5;

// 2. Exclusive check: no node should be both Band and
SoloMusician
MATCH (a:Artist:SoloMusician:Band)
RETURN a LIMIT 5;
```

Relations

```
// INCLUDES
UNWIND $relRecords AS relRecord
MATCH (source: `Festival` { `festival_id`:
toInteger(trim(relRecord.`festival_id`)) })
MATCH (target: `Stage` { `stage_id`:
toInteger(trim(relRecord.`stage_id`)) })
MERGE (source)-[r: `INCLUDES`]->(target);

// HAS_PERFORMANCE
UNWIND $relRecords AS relRecord
MATCH (source: `Stage` { `stage_id`:
toInteger(trim(relRecord.`stage_id`)) })
MATCH (target: `Performance`{
`performance_id`:toInteger(trim(relRecord.`performance_id`)) })
MERGE (source)-[r: `HAS_PERFORMANCE`]->(target);

// HOSTS
UNWIND $relRecords AS relRecord
MATCH (source: `Festival` { `festival_id`:
toInteger(trim(relRecord.`festival_id`)) })
MATCH (target: `Stand` { `stand_id`:
toInteger(trim(relRecord.`stand_id`)) })
MERGE (source)-[r: `HOSTS`]->(target);

// SELLS_PRODUCT
UNWIND $relRecords AS relRecord
MATCH (source: `Stand` { `stand_id`:
toInteger(trim(relRecord.`stand_id`)) })
MATCH (target: `Product` { `barcode`:
toInteger(trim(relRecord.`Barcode`)) })
MERGE (source)-[r: `SELLS_PRODUCT`]->(target);

// BOUGHT_PRODUCT
UNWIND $relRecords AS relRecord
MATCH (source: `Customer`{ `customer_id`:
toInteger(trim(relRecord.`customer_id`)) })
MATCH (target: `Product` { `barcode`:
toInteger(trim(relRecord.`barcode`)) })
MERGE (source)-[r: `BOUGHT_PRODUCT`]->(target);

// PURCHASED_TICKET
UNWIND $relRecords AS relRecord
MATCH (source: `Customer`{ `customer_id`:
toInteger(trim(relRecord.`customer_id`)) })
MATCH (target: `Ticket` { `ticket_id`:
toInteger(trim(relRecord.`ticket_id`)) })
```

```
MERGE (source)-[r: `PURCHASED_TICKET`]->(target)
SET r.`paymentDate` = datetime(relRecord.`paymentDate`),
r.`paymentMethod` = relRecord.`paymentMethod`;

// SELLS
UNWIND $relRecords AS relRecord
MATCH (source: `Festival`{ `festival_id`:
toInteger(trim(relRecord.`festival_id`)) })
MATCH (target: `Ticket` { `ticket_id`:
toInteger(trim(relRecord.`ticket_id`)) })
MERGE (source)-[r: `SELLS`]->(target);

// PERFORMS_IN (SoloMusician)
UNWIND $relRecords AS relRecord
MATCH (source: `SoloMusician`{ `artist_id`:
toInteger(trim(relRecord.`artist_id`)) })
MATCH (target: `Performance` { `performance_id`:
toInteger(trim(relRecord.`performance_id`)) })
MERGE (source)-[r: `PERFORMS_IN`]->(target);

// PERFORMS_IN (Band)
UNWIND $relRecords AS relRecord
MATCH (source: `Band` { `artist_id`:
toInteger(trim(relRecord.`artist_id`)) })
MATCH (target: `Performance`{ `performance_id`:
toInteger(trim(relRecord.`performance_id`)) })
MERGE (source)-[r: `PERFORMS_IN`]->(target);
```

Queries

```
// Q1 - Create new SoloMusician (Aurora Luna) and link to existing
Performance “Neon Echoes”.

MATCH (p:Performance {title: "Neon Echoes"})
CREATE (a:Artist:SoloMusician {
artist_id: 4051,
stageName: "Aurora Luna",
country: "Italy",
genre: "Electronic",
fullName: "Aurora Bianchi",
nationality: "Italian",
instrument: "synth"
})
CREATE (a)-[:PERFORMS_IN]->(p)
RETURN a, p;


// Q2 - Create new Stage “Eco Dome Milano” and connect it to
Festival “Milano Soundwave”.

MATCH (f:Festival {name: "Milano Soundwave"})
CREATE (s:Stage {
stage_id: 2051,
name: "Eco Dome Milano",
maxCapacity: 7000,
locationDescription: "Parco Sempione - area nord"
})
CREATE (f)-[:INCLUDES]->(s)
RETURN f.name AS Festival, s.stage_id AS ID_Stage, s.name AS
Stage, s.maxCapacity AS MaxCap, s.locationDescription AS Location
ORDER BY ID_Stage;


// Q3 - Update VIP ticket prices (+10%) for Milano Soundwave and
show before/after values.

MATCH (:Festival {name:"Milano Soundwave"})-[:SELLS]->(t:Ticket
{type:"VIP"})
WITH t, t.price AS oldPrice
SET t.price = t.price * 1.10
RETURN
t.ticket_id AS Ticket_ID,
ROUND(oldPrice, 2) AS OldPrice,
ROUND(t.price, 2) AS NewPrice;
```

```
// Q4 - Update Product price: apply €2 discount to "Vegetarian
Sandwich" and show results.

MATCH (p:Product {name: "Vegetarian Sandwich"})
WITH p, p.price AS originalPrice
SET p.price = p.price - 2
RETURN
p.name AS Product,
ROUND(originalPrice,2) AS OriginalPrice,
ROUND(p.price,2) AS DiscountedPrice;


// Q5 - Delete old Festivals (before 31 Dec 2025) and clean up
orphan nodes (Stage, Performance, Stand, Ticket, Product).

MATCH (f:Festival)
WHERE date(f.startDate) < date("2025-12-31")
WITH collect(f) AS toDelete, collect(f.name) AS deletedNames
CALL {
WITH toDelete
UNWIND toDelete AS fest
DETACH DELETE fest

}
CALL {
// cleanup orphans: Stage, Performance, Stand, Ticket, Product
WITH 1 AS _
CALL {
WITH _
MATCH (n:Stage)
WHERE NOT (n)--()
WITH collect(n) AS ns
FOREACH (x IN ns | DETACH DELETE x)
}
CALL {
WITH _
MATCH (n:Performance)
WHERE NOT (n)--()
WITH collect(n) AS ns
FOREACH (x IN ns | DETACH DELETE x)
}
CALL {
WITH _
MATCH (n:Stand)
WHERE NOT (n)--()
WITH collect(n) AS ns
FOREACH (x IN ns | DETACH DELETE x)
}
```

```
CALL {
WITH _
MATCH (n:Ticket)
WHERE NOT (n)--()
WITH collect(n) AS ns
FOREACH (x IN ns | DETACH DELETE x)
}
CALL {
WITH _
MATCH (n:Product)
WHERE NOT (n)--()
WITH collect(n) AS ns
FOREACH (x IN ns | DETACH DELETE x)
}
RETURN 0 AS _
}
WITH deletedNames
RETURN
size(deletedNames) AS Deleted_Festivals_Count,
deletedNames AS Deleted_Festivals_Names;


// Q6 - Retrieve Italian Artists performing at Milano Soundwave
(≥2 nodes + conditions).

MATCH (f:Festival {name:"Milano
Soundwave"})-[:INCLUDES]->(:Stage)-[:HAS_PERFORMANCE]->(p:Performa
nce)<-[:PERFORMS_IN]-(a:Artist)
WHERE a.country = "Italy"
RETURN DISTINCT a.stageName AS Artist, a.genre AS Genre
ORDER BY Artist;


// Q7 - Retrieve Electronic Performances after 20/06/2025 on
Stages with capacity >10,000 (≥2 nodes + conditions).

WITH datetime("2025-06-20T00:00:00") AS fromDate
MATCH
(:Festival)-[:INCLUDES]->(s:Stage)-[:HAS_PERFORMANCE]->(p:Performa
nce)
WHERE p.genre = "Electronic"
AND p.startTime > fromDate
AND s.maxCapacity > 10000
RETURN p.performance_id AS performance_id, p.title AS title,
p.startTime AS start, s.name AS stage, s.maxCapacity AS capacity
ORDER BY start;
```

```
// Q8 - Top 5 Most Active Stands (≥2 nodes + aggregation, no
WITH).

MATCH (s:Stand)-[:SELLS_PRODUCT]->(prod:Product)
RETURN s.standName AS Stand, COUNT(prod) AS ProductsSold
ORDER BY ProductsSold DESC LIMIT 5;


// Q9 - Ticket Economy at Roma Indie Fest: average ticket price
per type (≥2 nodes + aggregation, no WITH).

MATCH (f:Festival {name:"Roma Indie Fest"})-[:SELLS]->(t:Ticket)
RETURN t.type AS type, round(avg(t.price),2) AS avgPrice, count(*)
AS tickets
ORDER BY avgPrice DESC;


// Q10 - Food & Drink Fans: customers who bought both Food and
Drink items (≥2 nodes + conditions + 1 WITH).

MATCH (c:Customer)-[:BOUGHT_PRODUCT]->(pf:Product)
WHERE pf.category = "Food"
WITH c, COLLECT(pf.name) AS foods
MATCH (c)-[:BOUGHT_PRODUCT]->(pd:Product)
WHERE pd.category = "Drink"
RETURN c.fullName AS Customer, foods AS Food_Products,
COLLECT(pd.name) AS Drink_Products;


// Q11 - Frequent Buyers: customers with ≥2 distinct tickets (≥2
nodes + conditions + 1 WITH).

MATCH (c:Customer)-[:PURCHASED_TICKET]->(t:Ticket)
WITH c, count(DISTINCT t) AS bought
WHERE bought >= 2
RETURN c.customer_id AS customer_id, c.fullName AS customer,
bought AS ticketsBought
ORDER BY ticketsBought DESC, customer;
```

```
// Q12 - Festival Tourers: artists performing in ≥2 different
festivals (≥3 nodes + conditions + multiple WITH).

MATCH (a:Artist)-[:PERFORMS_IN]->(p:Performance)
MATCH (p)<-[:HAS_PERFORMANCE]-(:Stage)<-[:INCLUDES]-(f:Festival)
WITH a, COLLECT(DISTINCT f.name) AS festivals
WITH a, festivals, SIZE(festivals) AS numFest
WHERE numFest >= 2
RETURN a.stageName AS Artist, numFest AS Festivals, festivals
ORDER BY numFest DESC;

// Q13 - Longest Performance per Festival: compute performance
durations and return the longest one (≥3 nodes + conditions +
multiple WITH).


MATCH
(f:Festival)-[:INCLUDES]->(:Stage)-[:HAS_PERFORMANCE]->(p:Performa
nce)
WHERE p.startTime IS NOT NULL AND p.endTime IS NOT NULL
WITH f, p, duration.between(p.startTime, p.endTime) AS d
WITH f, p, (toInteger(d.seconds) / 60) AS mins
ORDER BY f.name, mins DESC, p.title
WITH f, collect({title:p.title, mins:mins})[0] AS top
RETURN
f.name AS festival,
top.title AS longestPerformance,
round(top.mins) AS minutes
ORDER BY minutes DESC, festival;


// Q14 - Variable-length path (min 2, max 10 hops).
Artist-Performance-Artist chaining.

MATCH (a1:Artist {stageName:"Laura Pausini"}),
(a2:Artist {stageName:"Marco Mengoni"})
MATCH path = (a1)-[:PERFORMS_IN*2..10]-(a2)
RETURN path
LIMIT 1;


// Q15 - Shortest path via only :PERFORMS_IN edges (alternates
Artist/Performance)

MATCH (a1:Artist {stageName:"Laura Pausini"}),
(a2:Artist {stageName:"Marco Mengoni"})
MATCH p = shortestPath( (a1)-[:PERFORMS_IN*2..10]-(a2) )
RETURN p;
```